

Home-usage-power-detector

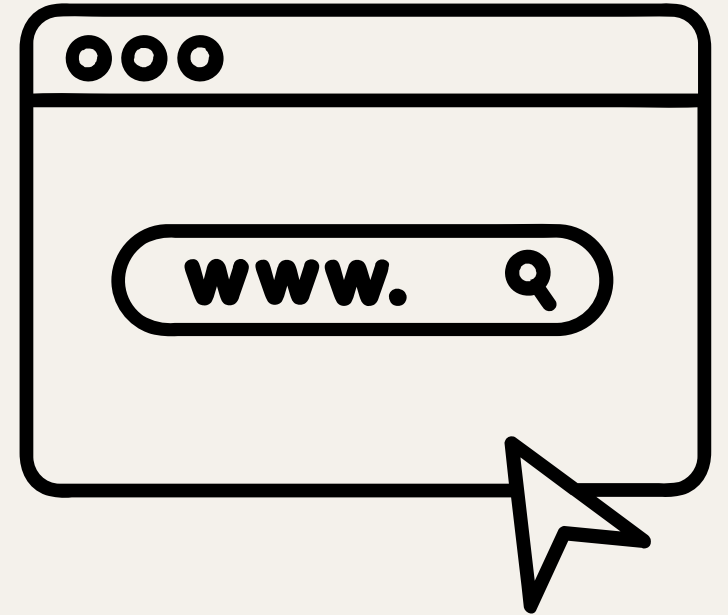
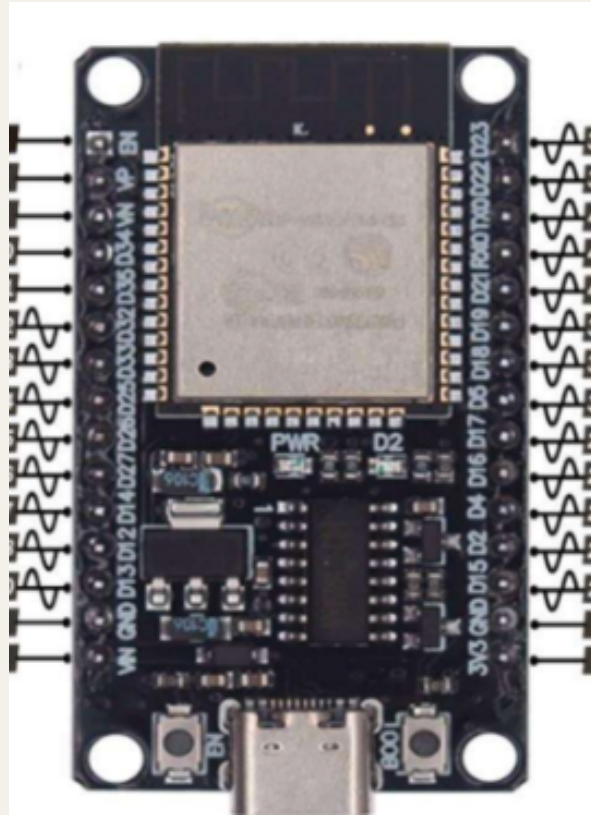
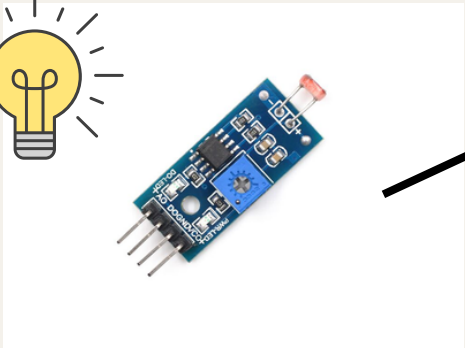
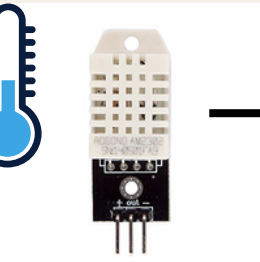
By G01 Siratee Tanjoy

REACH OUT

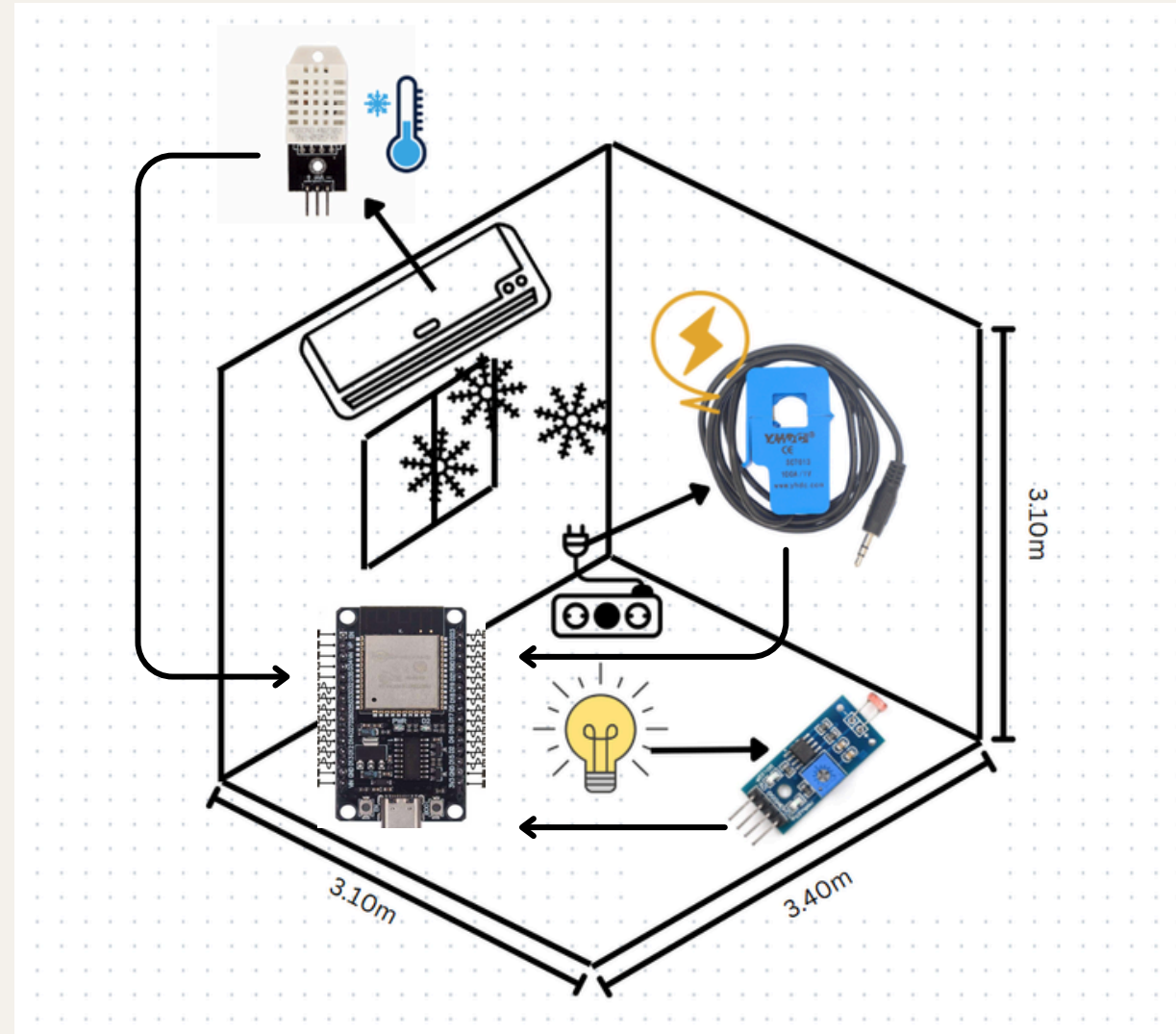
Project Requirement

- BCG or ESG relate (Topic: Smart home usage power detector)
- new self-study (CT sensor, DHT22, LDR sensor, real time website)
- version control (GitHub)
- RTOS (Use Mutex to control task)

Overview



Overview Design



Detail/Requirement

Detail

- ใช้ LDR Photoresistor Sensor ตรวจจับความเข้มแสงเพื่อแสดงสถานะว่าภายในห้อง สว่าง หรือ มืด
- สามารถ เปิด-ปิด ไฟผ่านหน้าเว็บได้ (ver.จำลอง) และคำนวณค่าไฟตามช่วงที่เปิด
- ใช้ DHT22 ตรวจจับ อุณหภูมิ และ ความชื้น เพื่อแสดงสถานะว่าภายในห้อง ร้อน หรือ เย็น
- สามารถ เปิด-ปิด เครื่องปรับอากาศผ่านหน้าเว็บได้ (ver.จำลอง) และคำนวณค่าไฟตามช่วงที่เปิด
- ใช้ CT Sensor ตรวจจับกระแสไฟไหลผ่านของสายไฟเพื่อแสดงคำนวณการใช้ไฟฟ้า และ ค่าไฟ

Requirement

- ห้องขนาดใดก็ได้ที่มี Internet เข้าถึง
- สายไฟที่อยู่ระหว่างแหล่งจ่ายไฟกับเครื่องใช้ไฟฟ้าต้องเป็นสายแบบ Single-Core ถ้าเป็นแบบ Multi-Core ต้องสามารถเข้าถึงสายต่างๆได้

Specification

Hardware

- ESP32 Dev Board CH340 x1
- Expansion Board ESP-32 WROOM 30pin GPIO TYPE-C USB x1
- LDR Photoresistor Sensor Module x1
- DHT22 AM2302 Module x1
- CT Sensor 100A SCT-013-000 x2
- 10uf Capacitor x2
- 10k Ohm Resistor x4
- 33 Ohm Resistor x2
- TRRS 3.5mm Jack Breakout x2 (Optional)
- Breadboard x1

Specification

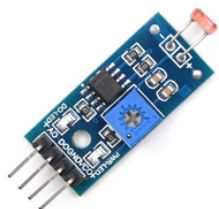
Software

- **Framework:** Arduino
- **Version control:** GitHub
- adafruit/DHT sensor library@^1.4.6
 - #include <DHT.h>
 - #include <Adafruit_Sensor.h>
- openenergymonitor/EmonLib@^1.1.0
 - #include "EmonLib.h"
- esphome/ESPAsyncWebServer-esphome@^3.3.0
 - #include <ESPAsyncWebServer.h>
- #include <Arduino.h>
- #include <WiFi.h>
- #include "index.h"
- task processing
- Mutex key

Architectural Design



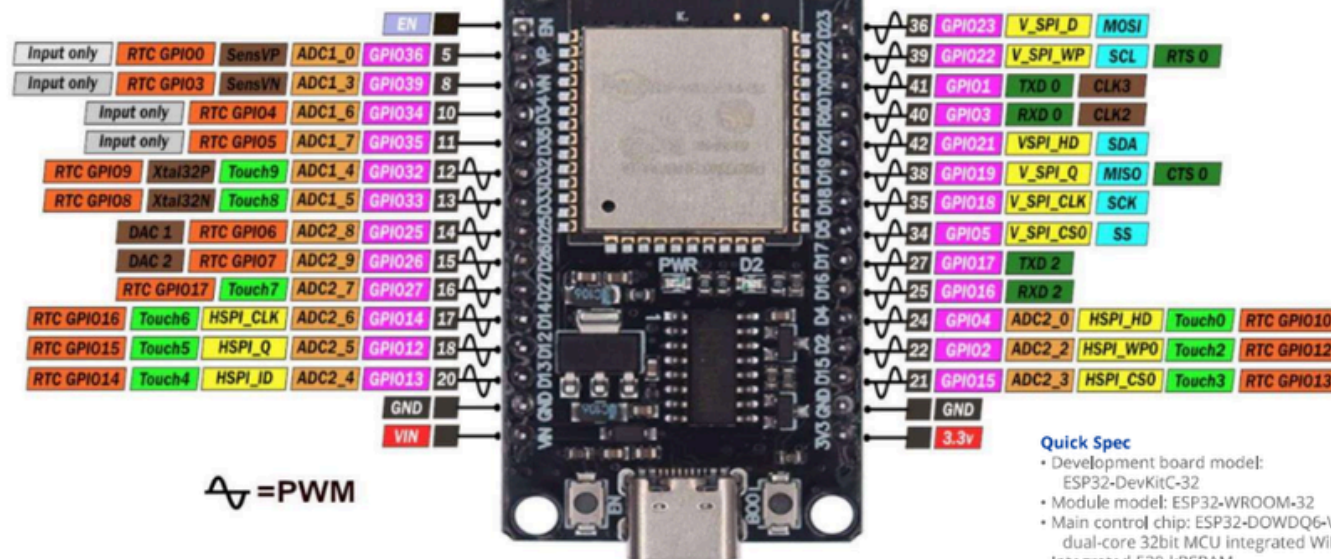
- VN(GPIO39)
- GPIO35



GPIO34

Pin Layout

ESP32-DevKitC



ESP32 Dev Board CH340 - USB-C

This is the ESP32S-DEV development board based on ESP32-WROOM-32, features WiFi + Bluetooth connectivity, onboard USB CH340 and keys. All the I/O pins of ESP32-WROOM-32 module are accessible via the extension headers. The board has 2x15pin extension headers, which breakout all the I/O pins of the module and 2x keys, used as reset or user-defined

- Input only GPIO Input only
- GPIO32 GPIO Input and Output
- DAC 1 Digital to Analog Converter
- ADC1_0 Analog to Digital Converter
- Touch0 Touch Sensor Input Channel
- RTC GPIO0 RTC Power Domain
- GND Ground
- 3.3v Power Rails (3V3 and 5V)

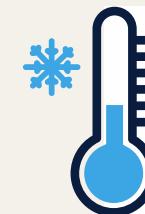
Quick Spec

- Development board model: ESP32-DevKitC-32
- Module model: ESP32-WROOM-32
- Main control chip: ESP32-D0WDQ6-V3 dual-core 32bit MCU integrated WiFi, Bluetooth,
- Integrated 520-kBSRAM, 448-kBROM16-kBSRAMinRTC
- External storage: 4MB
- USB driver chip: CH340C, with good system compatibility, higher download speed, and more stability.
- Support VIN external wide voltage input 5-12V power supply (battery version maximum 5.5V input) Support USB power supply, external 3.3V power supply, and VIN power supply three kinds of power supply
- Mode development board size: 52*28mm/weight about 9.5g
- Support ArduinoIDE mixly, mind+, Python and other programming software

ESP32 Dev Board CH340 - USB-C - Micro Robotics

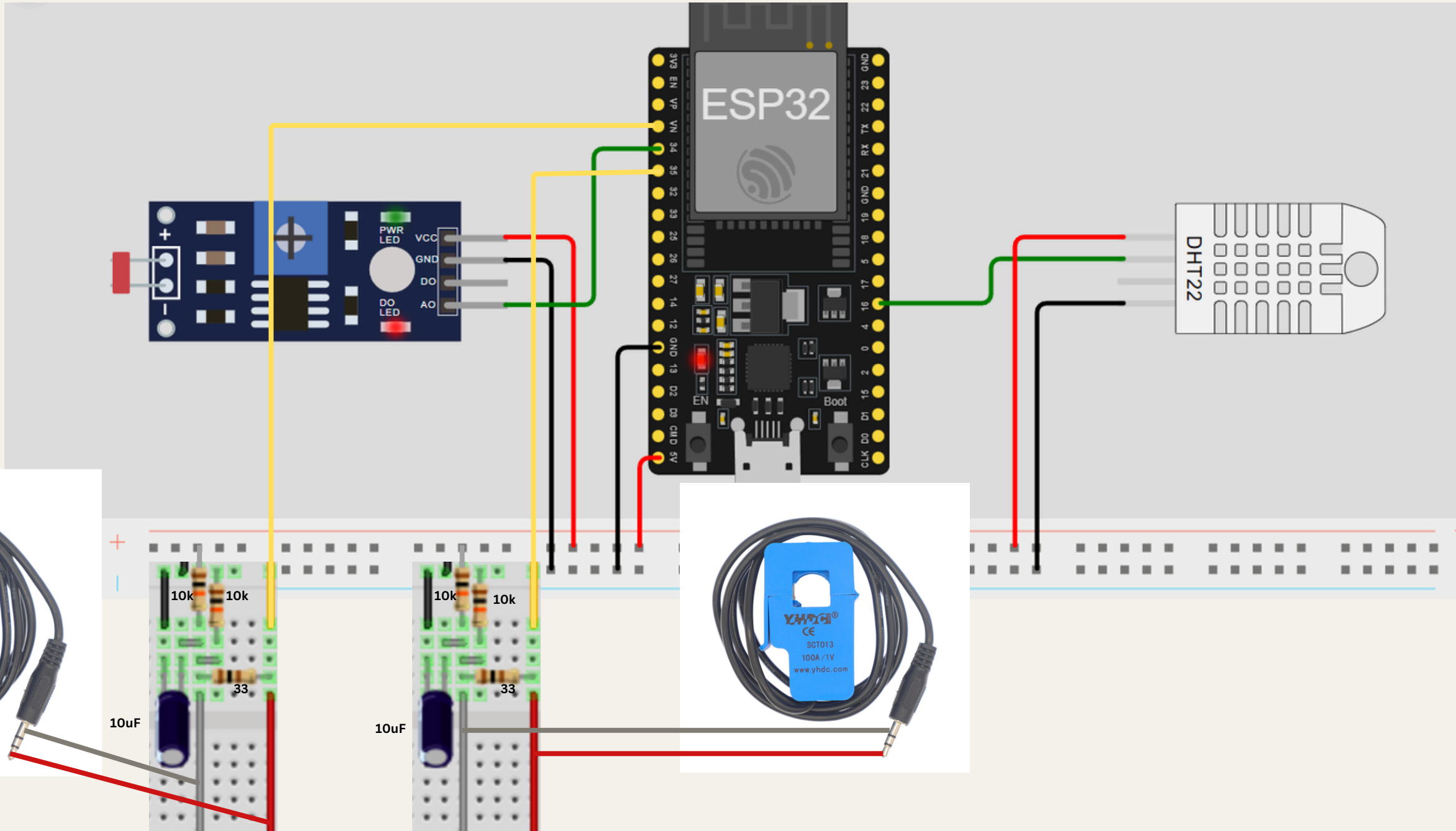
ESP32 TYPE-C USB ESP-32 Wi-Fi + Bluetooth Dual Core ESP32-DevKitC-32 ESP-WROOM - AliExpress 502

- VCC 3.3V or 5V
- GND

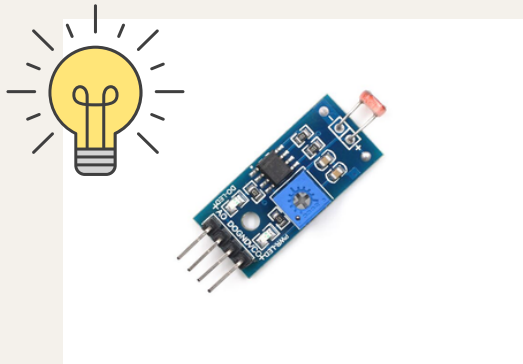


GPIO16

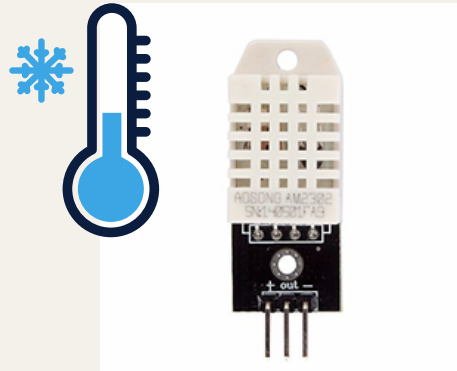
Architectural Design



Detailed Design



- get analog light value

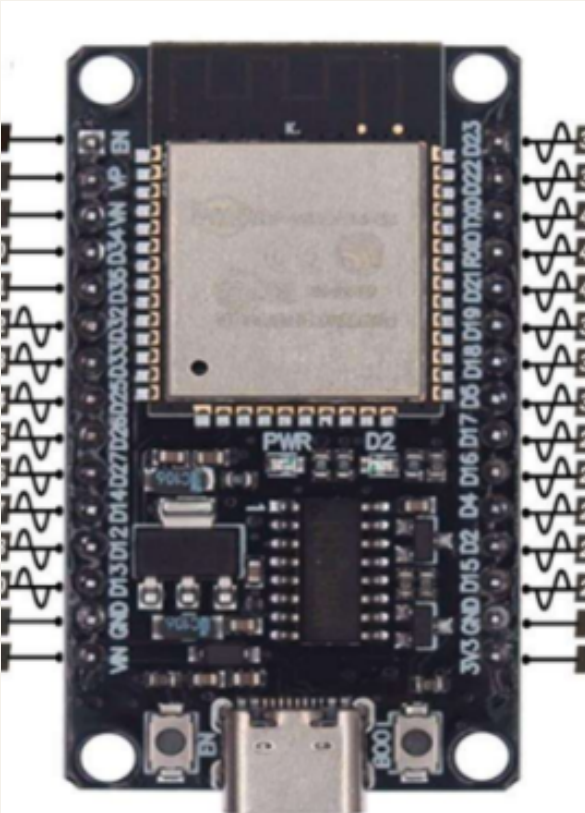


- get analog temperature value
- get analog humidity value

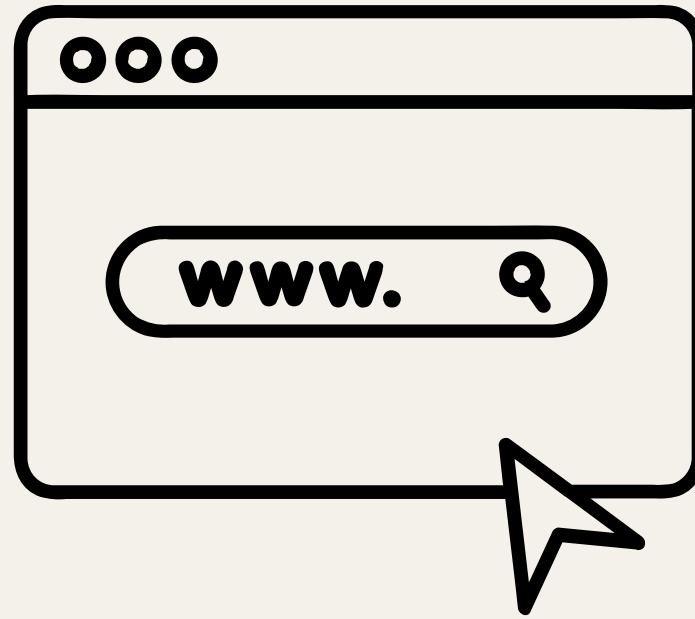


- get mean of analog current

Detailed Design

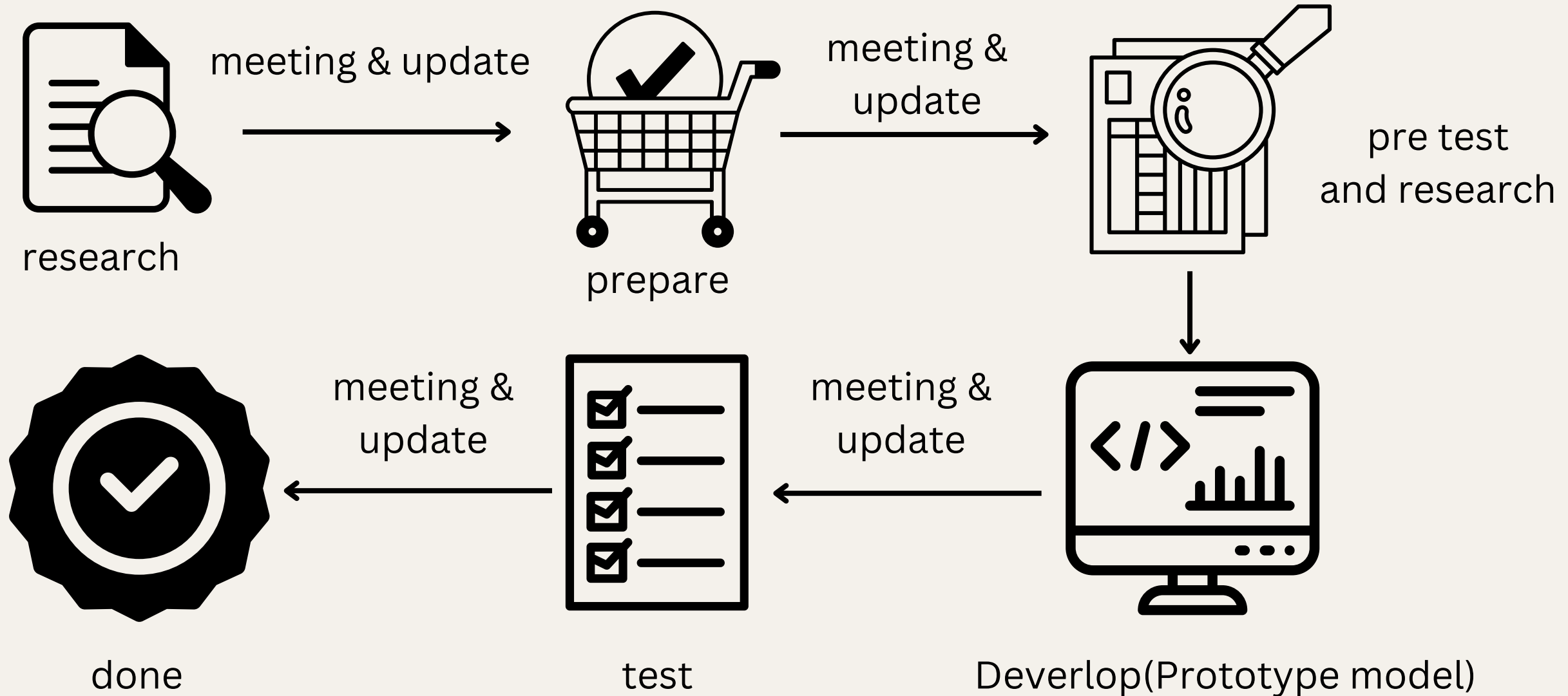


- get and calculate values from sensors
- send values to website
- host the website
- get control requests from website
- processes by tasks, each task has 22.5 secs delay

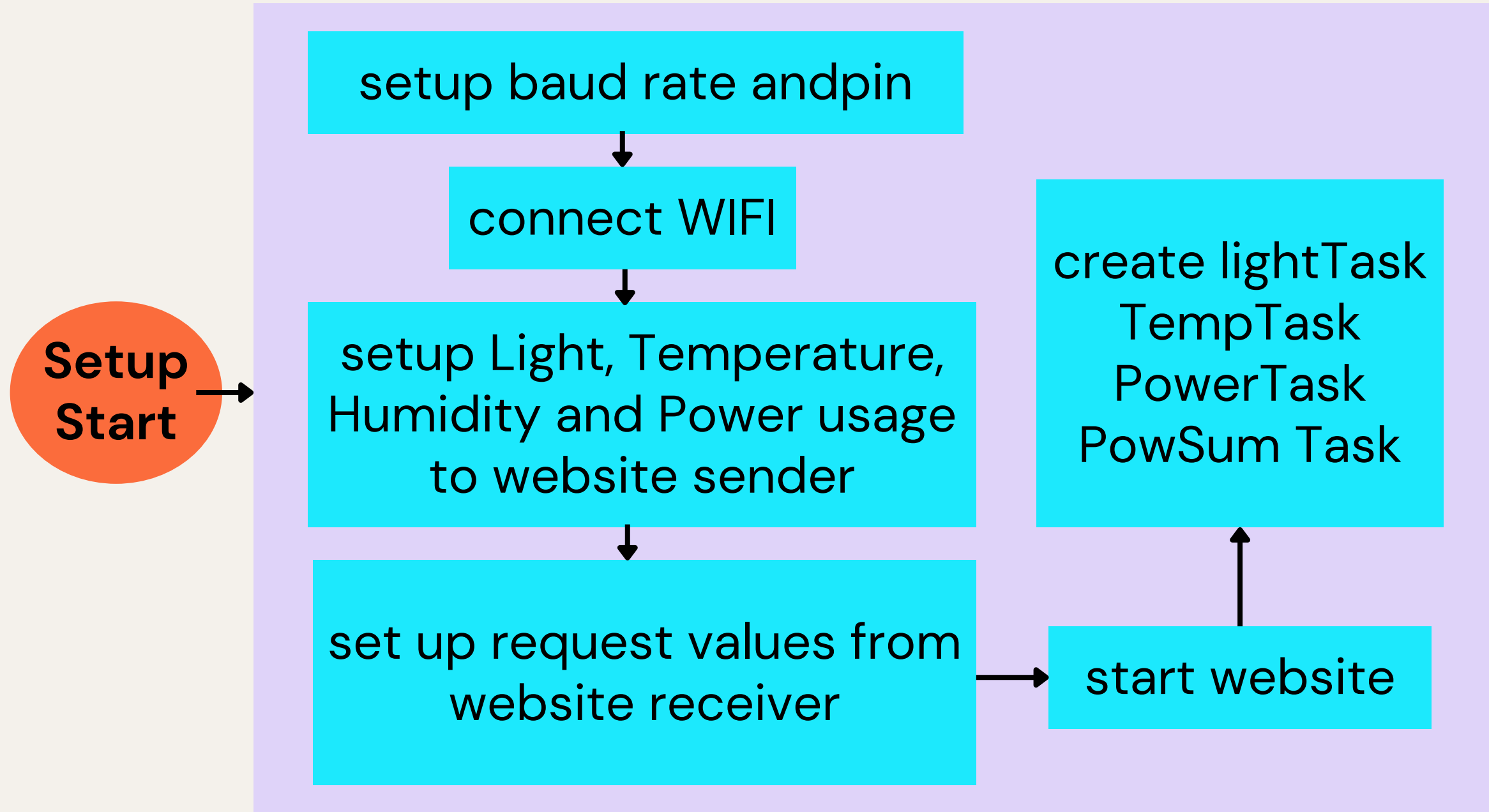


- get values from ESP32 and display
- sent control request
- refresh every 22.6 secs

Work Flow

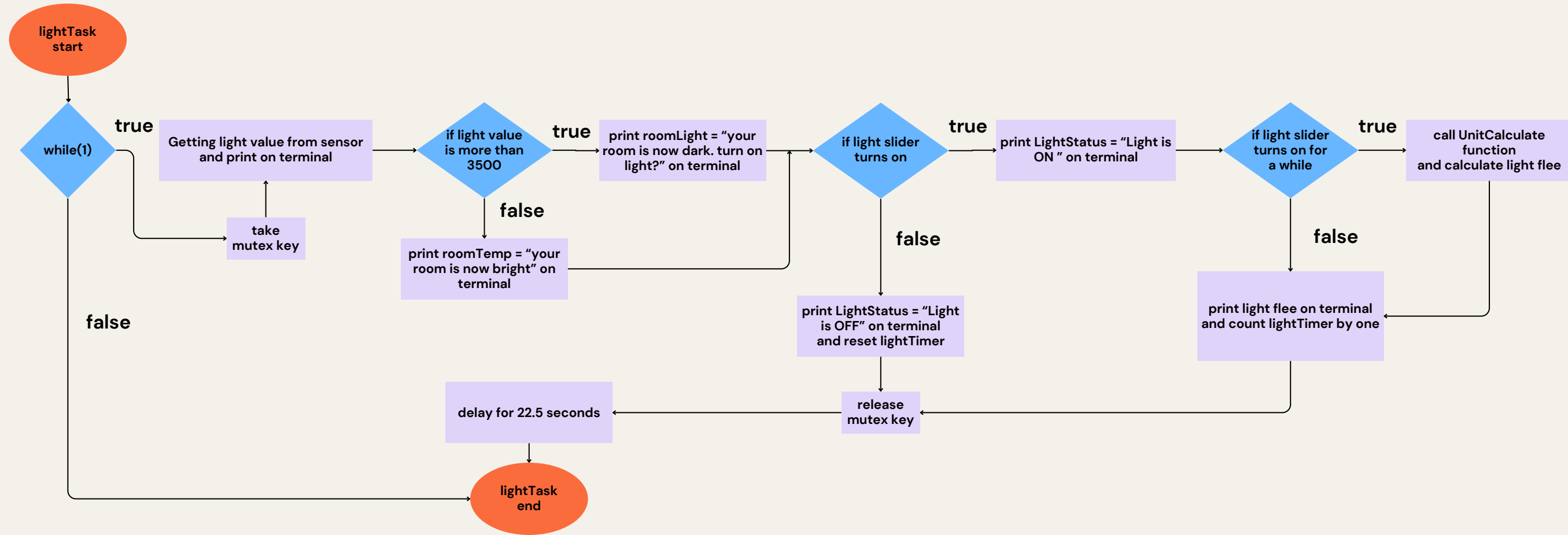


Flow Chart



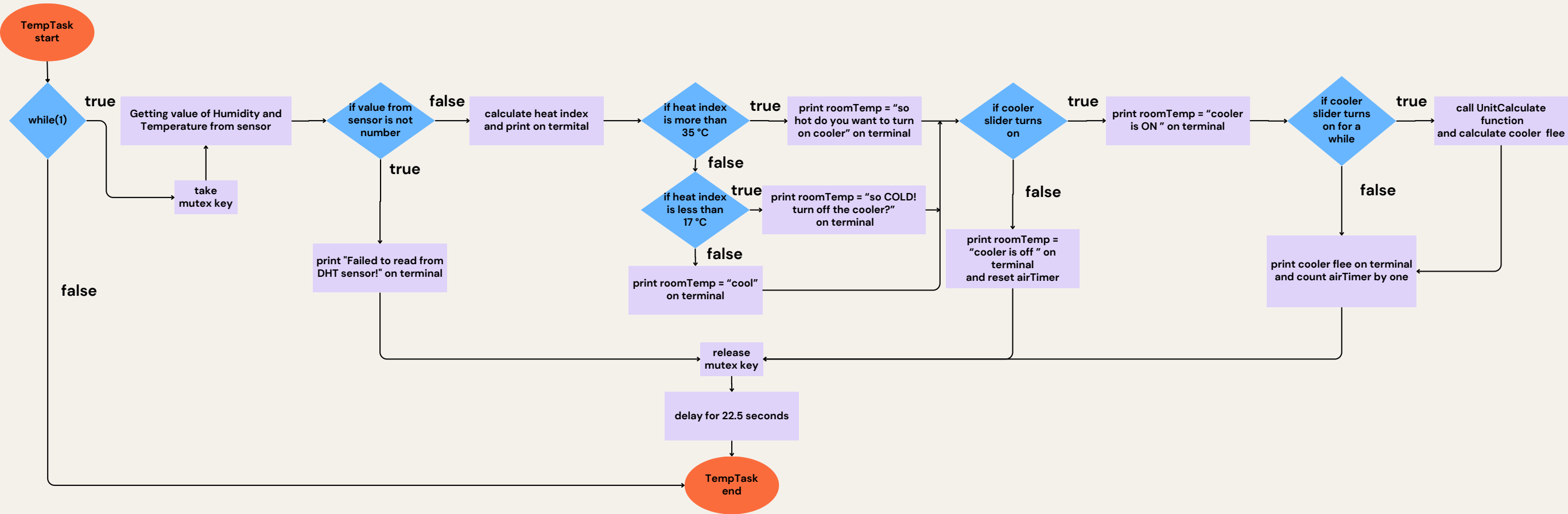
Flow Chart

lightTask



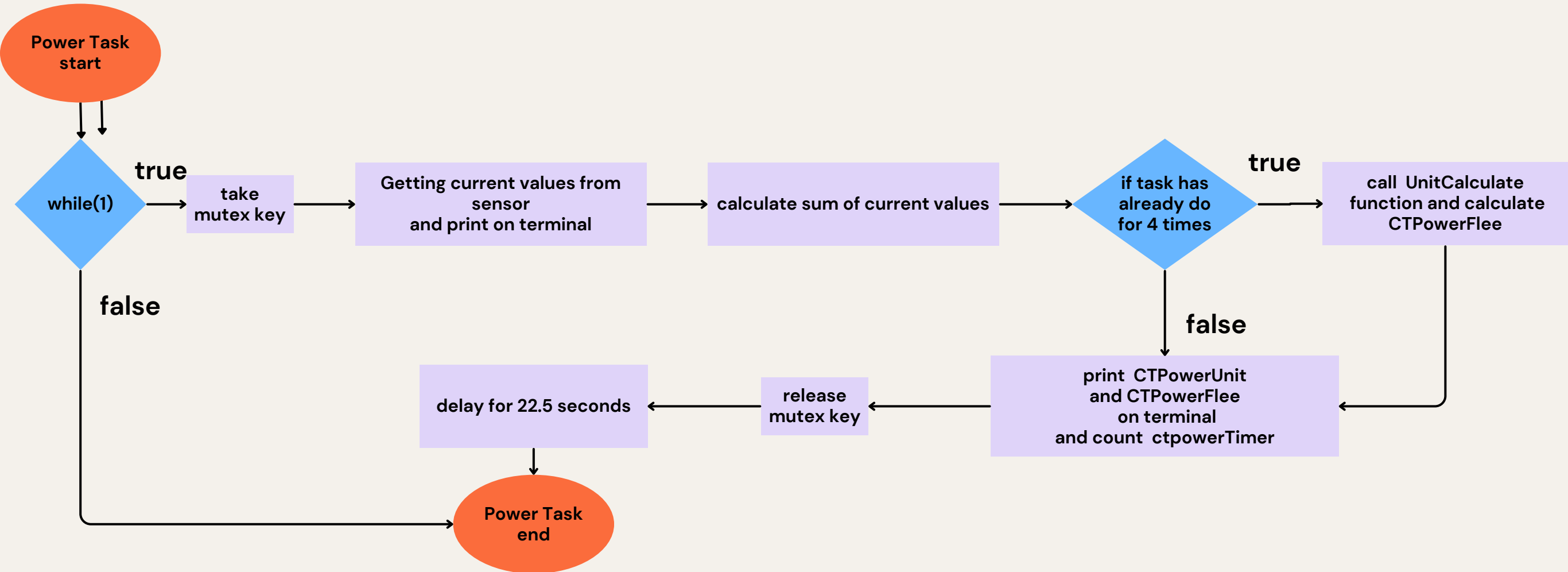
Flow Chart

Temp(Temperature)Task



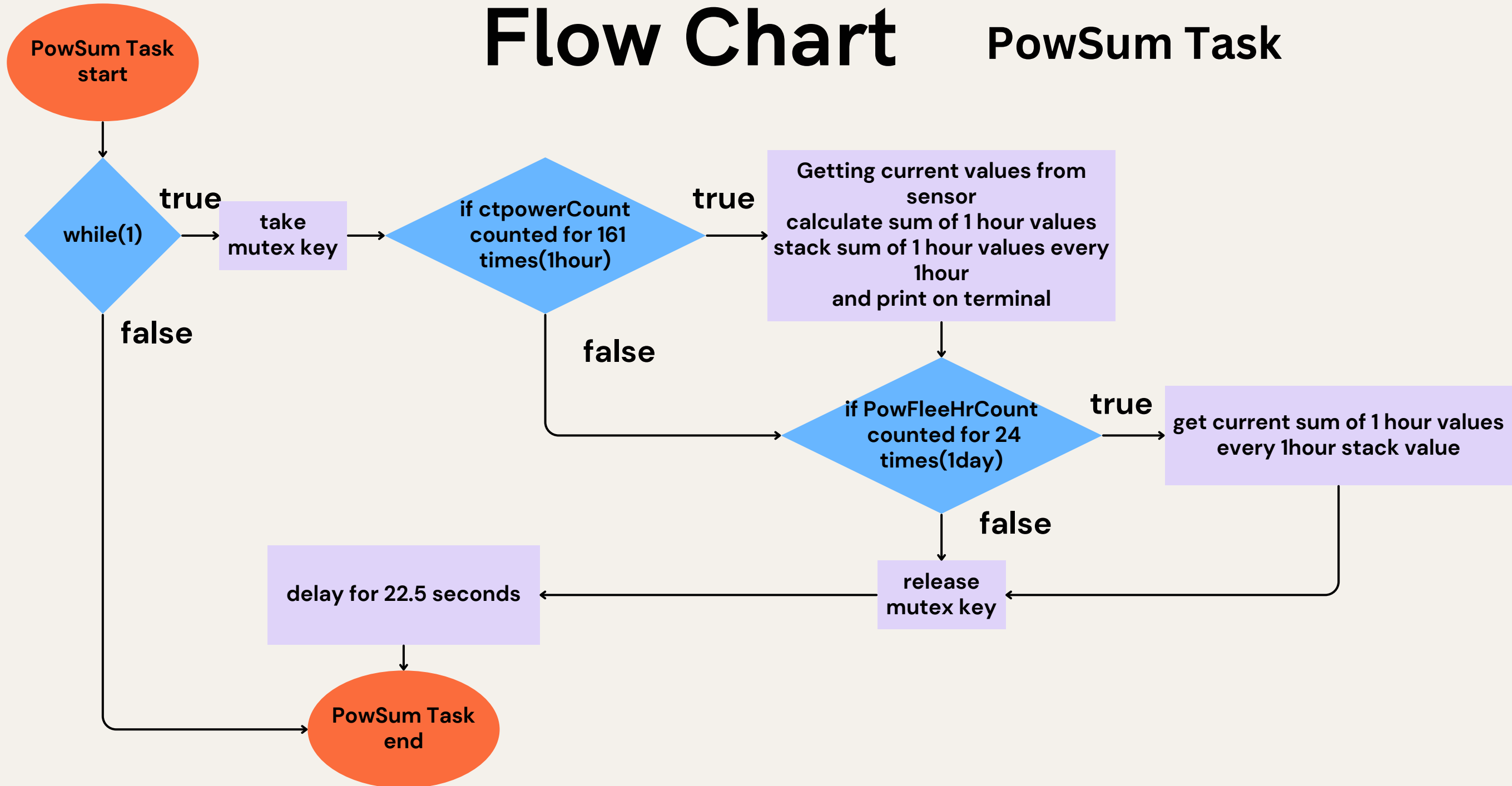
Flow Chart

Power Task



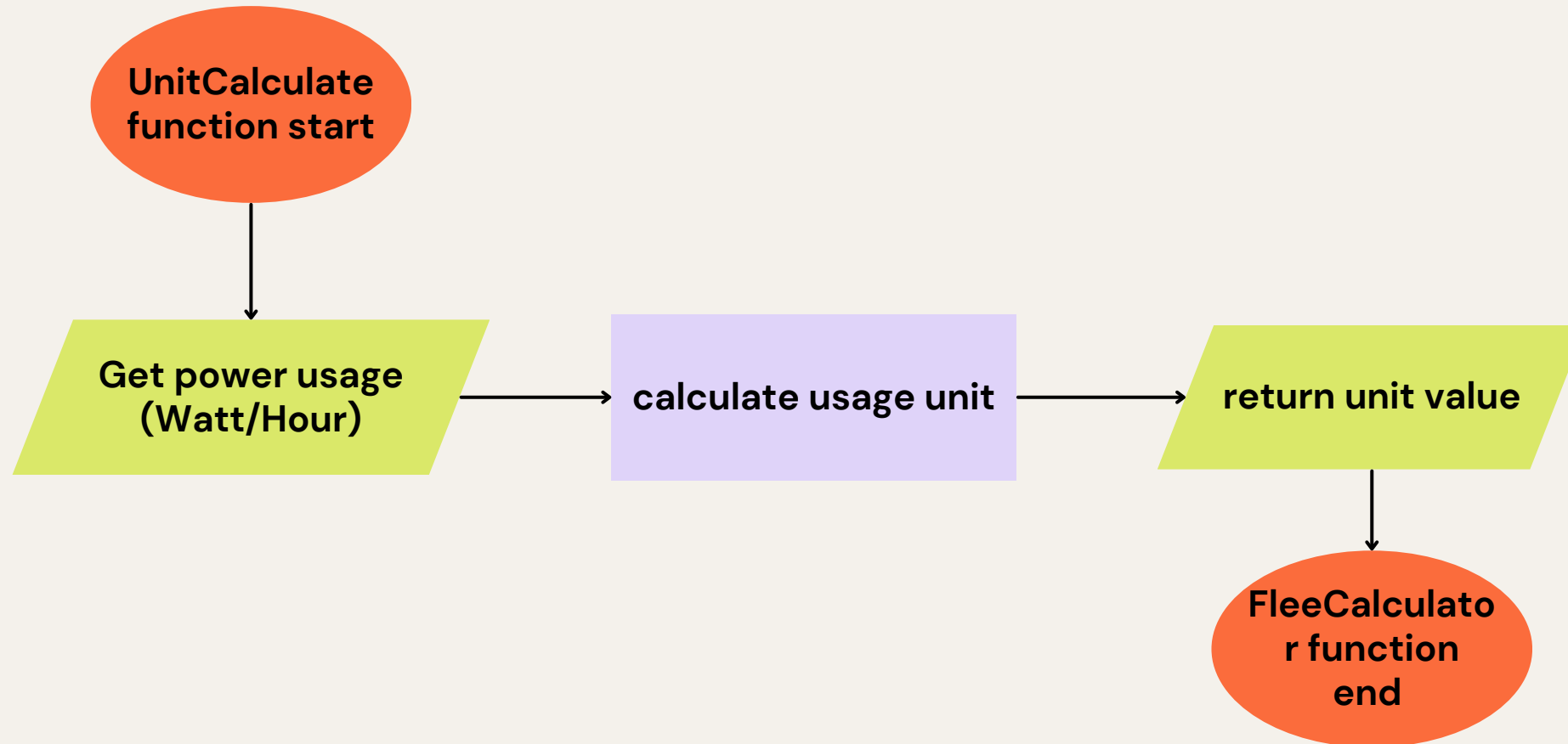
Flow Chart

PowSum Task



Flow Chart

UnitCalculate function



Flow Chart

light data sending

ESP32

index.h JS

refresh every

22.6 secs

HTML

send light status
light flee
light switch status

function lightMonitor
get light status
light flee
light switch status

show light status
light flee
light switch status

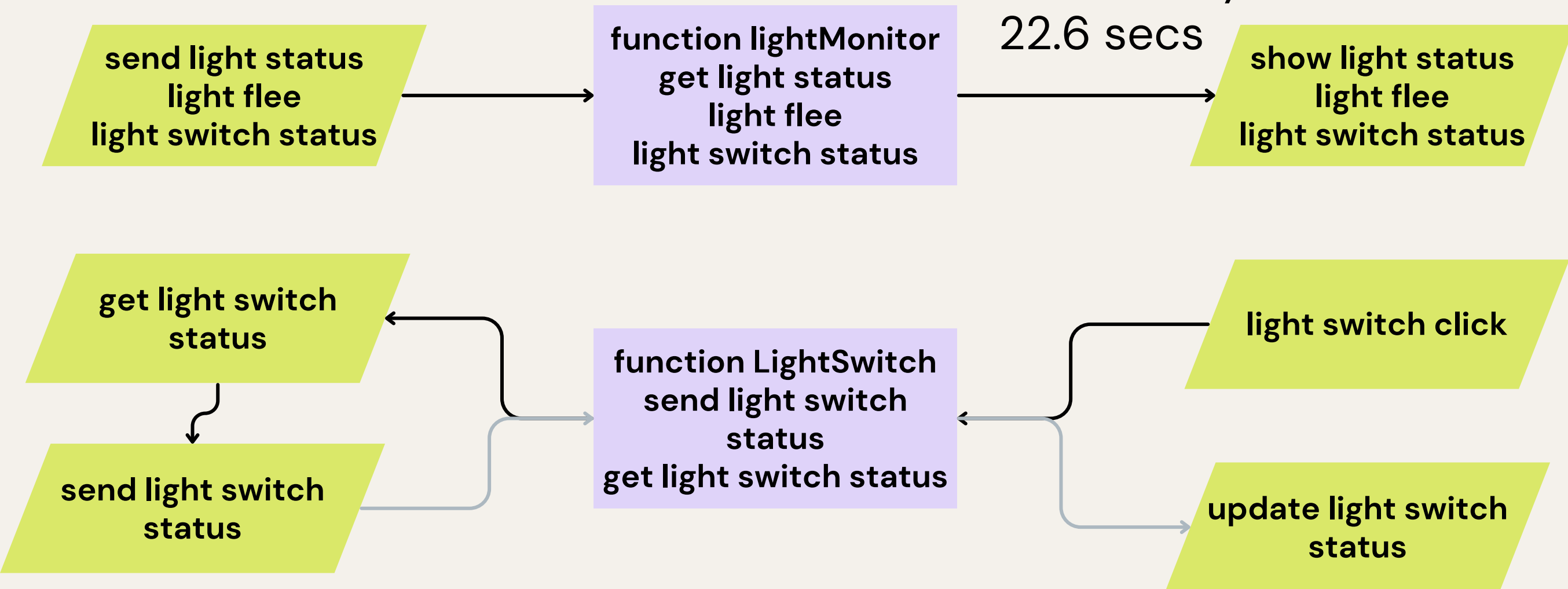
get light switch
status

send light switch
status

function LightSwitch
send light switch
status
get light switch status

light switch click

update light switch
status



Flow Chart

air data sending

ESP32

send temperature
heatIndex
heatStatus
cooler switch status

index.h JS

function temperature
get temperature
heatIndex
heatStatus
cooler switch status

refresh every
22.6 secs

HTML

show temperature
heatIndex
heatStatus
cooler switch status

get cooler switch
status

send cooler switch
status

function AirconSwitch
send cooler switch
status
get cooler switch
status

cooler switch click

update cooler
switch status

Flow Chart

ct power & power flee/hour data sending

ESP32

index.h JS

HTML

send CTPowerFlee

function CTPower
get CTPowerFlee

refresh every
22.6 secs

show CTPowerFlee

send
PowerFleePerHr

function PowFleePer
get PowerFleePerHr
PowerFleePerDay

refresh every
22.6 secs

show
PowerFleePerHr
PowerFleePerDay

send
PowerFleePerDay

Code

```
#include <Arduino.h>
#include <DHT.h>
#include <Adafruit_Sensor.h>
#include "EmonLib.h"
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#include "index.h"

#define DHTPIN 16 // Digital pin connected to the DHT sensor
#define DHTTYPE DHT22 // DHT 22 (AM2302), AM2321
DHT dht(DHTPIN, DHTTYPE);
int analogPin = 34, light = 0, lightTemp;
SemaphoreHandle_t xMutex;

int state = 0;

int LtaskPrio = 1, TtaskPrio = 1, lightCount = 0, airCount = 0, ctpowerCount = 0;
float lightUnit, powerFlee, lightPower = 46, CheckDelay = 22500;
float airUnit, usageUnit, airFlee, airPower = 782.81;
float CTPower, CTPowerUnit, CTPowerFlee;
float PowFleePerHr, PowFleePerHr_Temp;
String lightStatus, heatStatus;

float h, t, hic;

EnergyMonitor emon1;
EnergyMonitor emon2;

String airSliderValue = "OFF";
String lightSliderValue = "OFF";

const char* PARAM_INPUT = "value";

const char* WIFI_NAME = "Homewifi_2.4G";
const char* WIFI_PASSWORD = "No0955653261";
AsyncWebServer server(80);
```

ส่วนของ การนำเข้าlibrary และ
การประกาศตัวแปรเบื้องต้น

ส่วนของ UnitCalculate function

```
38 float UnitCalculate(float power){
39     usageUnit = power*(CheckDelay/3600000)/1000;
40     return usageUnit;
41 }
```

Code

ส่วนของ setup

กำหนด PIN และ เชื่อมต่อ WIFI

```
184 void setup(){
185   Serial.begin(9600);
186   pinMode(analogPin, INPUT);
187   emon1.current(35, 111.1);
188   emon2.current(39, 111.1);
189   dht.begin();
190
191   // Connect to Wi-Fi
192   WiFi.begin(WIFI_NAME, WIFI_PASSWORD);
193   while (WiFi.status() != WL_CONNECTED) {
194     delay(1000);
195     Serial.println("Connecting to WiFi...");
196   }
197   Serial.println("Connected to WiFi");
198
199   // Print the ESP32's IP address
200   Serial.print("ESP32 Web Server's IP address: ");
201   Serial.println(WiFi.localIP());
202 }
```

สร้าง Mutex และ Task

```
312 xMutex = xSemaphoreCreateMutex();
313
314 xTaskCreate(lightTask, "lightTask", 2048, NULL, LtaskPrio, NULL);
315 xTaskCreate(TempTask, "TempTask", 4096, NULL, TtaskPrio, NULL);
316 xTaskCreate(Power, "Power", 8192, NULL, TtaskPrio, NULL);
317 xTaskCreate(PowPerHr, "PowPerHr", 4096, NULL, LtaskPrio, NULL);
318
319 }
```

สร้างหน้า HTML

```
203 // Serve the HTML page from the file
204 server.on("/", HTTP_GET, [](AsyncWebServerRequest* request) {
205   Serial.println("ESP32 Web Server: New request received:"); // for debugging
206   Serial.println("GET /"); // for debugging
207
208   request->send(200, "text/html", webpage);
209 });
```

เปิด webserver

```
308
309 // Start the server
310 server.begin();
311
```

Code

ส่วนของ lightTask

```
43 void lightTask(void *param){
44     while(1){
45         xSemaphoreTake( xMutex,portMAX_DELAY);
46         light = analogRead(analogPin);
47         Serial.println(light);
48         if(light < 3800){
49             lightStatus = "Bright!";
50             Serial.println(lightStatus);
51         }
52         else{
53             if(lightSliderValue == "OFF")
54                 lightStatus = "Dark! turn on the light?";
55             Serial.println(lightStatus);
56         }
57
58         if(lightSliderValue == "ON"){
59             Serial.printf("\n\t\tlight is %s", lightSliderValue);
60             if(lightCount > 0){
61                 lightUnit += UnitCalculate(lightPower);
62                 powerFlee = lightUnit * 3.96;
63             }
64             Serial.printf("%.4f\n",lightUnit);
65             Serial.printf("%.4f\n",powerFlee);
66             lightCount++;
67         }
68         else if(lightSliderValue == "OFF"){
69             Serial.printf("\n\t\tlight is %s", lightSliderValue);
70             lightCount = 0;
71         }
72
73         xSemaphoreGive( xMutex ); //release key(xMutex)
74         vTaskDelay(pdMS_TO_TICKS(CheckDelay));
75     }
76 }
```

ส่วนของ lightTask ใน setup() เพื่อส่งข้อมูลไปหน้าเว็บ

```
211 // Define a route to get the light data
212 server.on("/light", HTTP_GET, [](AsyncWebServerRequest* request) {
213     Serial.println("ESP32 Web Server: New request received:"); // for debugging
214     Serial.println("GET /light"); // for debugging
215     request->send(200, "text/plain", lightStatus);
216 });
217
218 server.on("/LightSwitchStatus", HTTP_GET, [](AsyncWebServerRequest* request) {
219     Serial.println("ESP32 Web Server: New request received:"); // for debugging
220     Serial.println("GET /LightSwitchStatus"); // for debugging
221     request->send(200, "text/plain", lightSliderValue);
222 });
223
224 server.on("/lightFlee", HTTP_GET, [](AsyncWebServerRequest* request) {
225     Serial.println("ESP32 Web Server: New request received:"); // for debugging
226     Serial.println("GET /lightFlee"); // for debugging
227     String lightFlee = String(powerFlee, 4);
228     request->send(200, "text/plain", lightFlee);
229 });
```

Code

ส่วนของ lightTask

ส่วนของ lightTask ใน setup() เพื่อรับข้อมูลจากหน้าเว็บ

```
295 server.on("/lightSlider", HTTP_GET, [] (AsyncWebServerRequest *request) {
296     String LightInputMessage;
297     // GET input1 value on <ESP_IP>/slider?value=<inputMessage>
298     if (request->hasParam(PARAM_INPUT)) {
299         LightInputMessage = request->getParam(PARAM_INPUT)->value();
300         lightSliderValue = LightInputMessage;
301     }
302     else {
303         LightInputMessage = "No message sent";
304     }
305     Serial.println(LightInputMessage);
306     request->send(200, "text/plain", "OK");
307 });
```

ส่วนของ lightTask ใน index.h js เพื่อรับข้อมูลจากESP32

```
128 <script>
129     function lightMonitor() {
130         fetch("/light")
131             .then(response => response.text())
132             .then(data => {
133                 document.getElementById("light").textContent = data;
134             });
135
136         fetch("/lightFlee")
137             .then(response => response.text())
138             .then(data => {
139                 document.getElementById("lightFlee").textContent = data;
140             });
141
142         fetch("/LightSwitchStatus")
143             .then(response => response.text())
144             .then(data => {
145                 document.getElementById("LightSwitchStatus").textContent = data;
146             });
147     }
148
149     lightMonitor();
150     setInterval(lightMonitor, 22600); // Update temperature every 4 seconds
```

Code

ส่วนของ lightTask

ส่วนของ lightTask ใน index.h js เพื่อส่งข้อมูลไปESP32

```
153 function LightSwitch() {
154     var LightCheckBox = document.getElementById("LightSwitchCheck");
155     var Lightxhr = new XMLHttpRequest();
156     if (LightCheckBox.checked == true){
157         Lightxhr.open("GET", "/lightSlider?value=ON", true);
158     } else {
159         Lightxhr.open("GET", "/lightSlider?value=OFF", true);
160     }
161     Lightxhr.send();
162
163     fetch("/LightSwitchStatus")
164         .then(response => response.text())
165         .then(data => {
166             document.getElementById("LightSwitchStatus").textContent = data;
167         });
168 }
```

ส่วนของการแสดงผล lightTask ใน index.h

```
89 <body>
90     <h1>Home power usage monitor</h1>
91     <i class="fa fa-lightbulb-o" style="font-size:48px; color:#ffca04;"></i>
92     <h2>
93         Your room is now <span style="color: red;"><span id="light">Loading...</span></span>
94     </h2>
95     <h2>
96         Light switch is turn <span style="color: red;"><span id="LightSwitchStatus">Loading...</span></span>
97         <label class="switch">
98             <input type="checkbox" id="LightSwitchCheck" onclick="LightSwitch()">
99             <span class="slider round"></span>
100         </label>
101     </h2>
102     <h2>
103         Current Power Flee: <span style="color: red;"><span id="lightFlee">Loading...</span></span>THB
104     </h2>
105     <br>
```


Code

ส่วนของ TempTask

```
78 void TempTask(void *param){
79     while(1){
80         xSemaphoreTake( xMutex,portMAX_DELAY);
81         // Reading temperature or humidity takes about 250 milliseconds!
82         // Sensor readings may also be up to 2 seconds 'old' (its a very slow sensor)
83         h = dht.readHumidity();
84         // Read temperature as Celsius (the default)
85         t = dht.readTemperature();
86
87         // Check if any reads failed and exit early (to try again).
88         if (isnan(h) || isnan(t)) {
89             Serial.println(F("Failed to read from DHT sensor!"));
90             return;
91         }
92
93         // Compute heat index in Celsius (isFahreheit = false)
94         hic = dht.computeHeatIndex(t, h, false);
95
96         Serial.print(F("\t\t\tHumidity: "));
97         Serial.print(h);
98         Serial.print(F("% Temperature: "));
99         Serial.print(t);
100        Serial.print(F(" C "));
101        Serial.print(F(" Heat index: "));
102        Serial.print(hic);
103        Serial.print(F(" C "));
104
105        if(hic >= 32){
106            if(airSliderValue == "OFF")
107                heatStatus = "so HOT! turn on the cooler?";
108            else heatStatus = " ";
109            Serial.println(heatStatus);
110        }
111        else if(hic < 17){
112            if(airSliderValue == "ON")
113                heatStatus = "so COLD! turn off the cooler?";
114            else heatStatus = " ";
115            Serial.println(heatStatus);
116        }
117        else{
118            heatStatus = "COOL!";
119            Serial.println(heatStatus);
120        }
121    }
122 }
```

```
99 Serial.print(t);
100 Serial.print(F(" C "));
101 Serial.print(F(" Heat index: "));
102 Serial.print(hic);
103 Serial.print(F(" C "));
104
105 if(hic >= 32){
106     if(airSliderValue == "OFF")
107         heatStatus = "so HOT! turn on the cooler?";
108     else heatStatus = " ";
109     Serial.println(heatStatus);
110 }
111 else if(hic < 17){
112     if(airSliderValue == "ON")
113         heatStatus = "so COLD! turn off the cooler?";
114     else heatStatus = " ";
115     Serial.println(heatStatus);
116 }
117 else{
118     heatStatus = "COOL!";
119     Serial.println(heatStatus);
120 }
121
122 if(airSliderValue == "ON"){
123     Serial.printf("\n\t\t\tair is %s", airSliderValue);
124     if(airCount > 0){
125         airUnit += UnitCalculate(airPower);
126         airFlee = airUnit * 3.96;
127     }
128     Serial.printf("AU: %.4f\n", airUnit);
129     Serial.printf("AF: %.4f\n", airFlee);
130     airCount++;
131 }
132 else if(airSliderValue == "OFF"){
133     Serial.printf("\n\t\t\tair is %s", airSliderValue);
134     airCount = 0;
135 }
136
137 xSemaphoreGive( xMutex ); //release key(xMutex)
138 vTaskDelay(pdMS_TO_TICKS(CheckDelay));
139 }
140 }
```


Code

ส่วนของ TempTask

ส่วนของ TempTask ใน setup() เพื่อส่งข้อมูลไปหน้าเว็บ

```
235 server.on("/temperature", HTTP_GET, [](AsyncWebServerRequest* request) {
236     Serial.println("ESP32 Web Server: New request received:"); // for debugging
237     Serial.println("GET /temperature"); // for debugging
238     float temperature = t;
239     String temperatureStr = String(temperature, 2);
240     request->send(200, "text/plain", temperatureStr);
241 });
242
243 server.on("/heatIndex", HTTP_GET, [](AsyncWebServerRequest* request) {
244     Serial.println("ESP32 Web Server: New request received:"); // for debugging
245     Serial.println("GET /heatIndex"); // for debugging
246     String heatIndexStr = String(hic, 2);
247     request->send(200, "text/plain", heatIndexStr);
248 });
249
250 server.on("/heatStatus", HTTP_GET, [](AsyncWebServerRequest* request) {
251     Serial.println("ESP32 Web Server: New request received:"); // for debugging
252     Serial.println("GET /heatStatus"); // for debugging
253     request->send(200, "text/plain", heatStatus);
254 });
255
256 server.on("/airStatus", HTTP_GET, [](AsyncWebServerRequest* request) {
257     Serial.println("ESP32 Web Server: New request received:"); // for debugging
258     Serial.println("GET /airStatus"); // for debugging
259     request->send(200, "text/plain", airSliderValue);
260 });
261
262 server.on("/airFlee", HTTP_GET, [](AsyncWebServerRequest* request) {
263     Serial.println("ESP32 Web Server: New request received:"); // for debugging
264     Serial.println("GET /airFlee"); // for debugging
265     float af = airFlee;
266     String AirFlee = String(af, 4);
267     request->send(200, "text/plain", AirFlee);
268 });
```

ส่วนของ TempTask ใน setup() เพื่อรับข้อมูลจากหน้าเว็บ

```
291
292 server.on("/airSlider", HTTP_GET, [] (AsyncWebServerRequest *request) {
293     String AirInputMessage;
294     // GET input1 value on <ESP_IP>/slider?value=<inputMessage>
295     if (request->hasParam(PARAM_INPUT)) {
296         AirInputMessage = request->getParam(PARAM_INPUT)->value();
297         airSliderValue = AirInputMessage;
298     }
299     else {
300         AirInputMessage = "No message sent";
301     }
302     Serial.println(AirInputMessage);
303     request->send(200, "text/plain", "OK");
304 });
```

Code

ส่วนของ TempTask ใน index.h js เพื่อรับข้อมูลจากESP32

```
170 function temperature() {
171   fetch("/temperature")
172     .then(response => response.text())
173     .then(data => {
174       document.getElementById("temperature").textContent = data;
175     });
176
177   fetch("/heatIndex")
178     .then(response => response.text())
179     .then(data => {
180       document.getElementById("heatIndex").textContent = data;
181     });
182
183   fetch("/heatStatus")
184     .then(response => response.text())
185     .then(data => {
186       document.getElementById("heatStatus").textContent = data;
187     });
188
189   fetch("/airFlee")
190     .then(response => response.text())
191     .then(data => {
192       document.getElementById("airFlee").textContent = data;
193     });
194
195   fetch("/airStatus")
196     .then(response => response.text())
197     .then(data => {
198       document.getElementById("airStatus").textContent = data;
199     });
200 }
```

ส่วนของ TempTask

ส่วนของการแสดงผล TempTask ใน index.h

```
106 <i class="fa fa-thermometer-half" style="font-size:48px; color:#059e8a;"></i>
107 <h2>
108   Temperature: <span style="color: red;"><span id="temperature">Loading...</span>&deg</span>
109   Heat Index: <span style="color: red;"><span id="heatIndex">Loading...</span>&deg</span>
110 </h2>
111 <h2>Your room is <span style="color: red;"><span id="heatStatus">Loading...</span></span>
112 <h2>
113   Air conditioner is <span style="color: red;"><span id="airStatus">Loading...</span></span>
114   <label class="switch">
115     <input type="checkbox" id="AirSwitchCheck" onclick="AirconSwitch()">
116     <span class="slider round"></span>
117   </label>
118 </h2>
119 <h2>
120   Air Flee: <span style="color: red;"><span id="airFlee">Loading...</span></span>THB
121 </h2>
122 <br>
```

ส่วนของ TempTask ใน index.h js เพื่อส่งข้อมูลไปESP32

```
205 function AirconSwitch() {
206   var AirCheckBox = document.getElementById("AirSwitchCheck");
207   var Airxhr = new XMLHttpRequest();
208   if (AirCheckBox.checked == true){
209     Airxhr.open("GET", "/airSlider?value=ON", true);
210   } else {
211     Airxhr.open("GET", "/airSlider?value=OFF", true);
212   }
213   Airxhr.send();
214
215   fetch("/airStatus")
216     .then(response => response.text())
217     .then(data => {
218       document.getElementById("airStatus").textContent = data;
219     });
220 }
```

Code

ส่วนของ Power Task

```
143 void Power(void *parameter){
144     while(1){
145         xSemaphoreTake( xMutex,portMAX_DELAY);
146         double Irms = emon1.calcIrms(1480); // Calculate Irms only
147         Serial.print(Irms*230.0);          // Apparent power
148         Serial.print(" ");
149         Serial.println(Irms);              // Irms
150
151         double Irmss = emon2.calcIrms(1480); // Calculate Irms only
152         Serial.print(Irmss*230.0);          // Apparent power
153         Serial.print(" ");
154         Serial.println(Irmss);              // Irms
155
156         CTPower = (Irms + Irmss)*230;
157
158         if(ctpowerCount > 3){
159             CTPowerUnit += UnitCalculate(CTPower);
160             CTPowerFlee = CTPowerUnit * 3.96;
161         }
162         Serial.printf("PU:%.4f\n",CTPowerUnit);
163         Serial.printf("PF:%.4f\n",CTPowerFlee);
164         ctpowerCount++;
165
166         Serial.println(uxTaskGetStackHighWaterMark(NULL));
167         xSemaphoreGive( xMutex ); //release key(xMutex)
168         vTaskDelay(pdMS_TO_TICKS(CheckDelay));
169     }
170 }
```

ส่วนของ PowSum Task

```
172 void PowSum(void *param){
173     while(1){
174         xSemaphoreTake( xMutex,portMAX_DELAY);
175         if(ctpowerCount % 161 == 0){
176             PowFleePerHr = powerFlee + airFlee + CTPowerFlee;
177             PowFleePerHr_Temp += PowFleePerHr;
178             PowFleeHrCount++;
179         }
180         if(PowFleeHrCount% 24 == 0){
181             PowFleePerDay = PowFleePerHr_Temp;
182         }
183         xSemaphoreGive( xMutex ); //release key(xMutex)
184         vTaskDelay(pdMS_TO_TICKS(CheckDelay));
185     }
186 }
```

Code

ส่วนของ Power Task และ PowSum Task

```
270 server.on("/CTPowerFlee", HTTP_GET, [](AsyncWebServerRequest* request) {  
271     Serial.println("ESP32 Web Server: New request received:"); // for debugging  
272     Serial.println("GET /CTPowerFlee"); // for debugging  
273     float CTPF = CTPowerFlee;  
274     String CTPFlee = String(CTPF, 4);  
275     request->send(200, "text/plain", CTPFlee);  
276 });
```

ส่วนของ Power Task ใน setup() เพื่อส่งข้อมูลไปหน้าเว็บ

ส่วนของ PowerSum Task ใน
setup() เพื่อส่งข้อมูลไปหน้าเว็บ

```
278 server.on("/PowerFleePerHr", HTTP_GET, [](AsyncWebServerRequest* request) {  
279     Serial.println("ESP32 Web Server: New request received:"); // for debugging  
280     Serial.println("GET /PowerFleePerHr"); // for debugging  
281     String PFleePerHr = String(PowFleePerHr, 4);  
282     request->send(200, "text/plain", PFleePerHr);  
283 });  
284  
285 server.on("/PowerFleePerDay", HTTP_GET, [](AsyncWebServerRequest* request) {  
286     Serial.println("ESP32 Web Server: New request received:"); // for debugging  
287     Serial.println("GET /PowerFleePerHr"); // for debugging  
288     String PFleePerDay = String(PowFleePerDay, 4);  
289     request->send(200, "text/plain", PFleePerDay);  
290 });
```

Code

ส่วนของ TempTask

ส่วนของการแสดงผล Power Task ใน index.h

```
123 <i class="fa fa-flash" style="font-size:48px;color:#00F0FF;"></i>  
124 <h2>Power usage Flee: <span style="color: red;"><span id="CTPowerFlee">Loading...</span></span>THB</h2>
```

ส่วนของการแสดงผล PowerSum Task ใน index.h

```
126 <h2>Power Flee/Hour: <span style="color: red;"><span id="PowerFleePerHr">Loading...</span></span>THB</h2>  
127 <h2>Power Flee/Day: <span style="color: red;"><span id="PowerFleePerDay">Loading...</span></span>THB</h2>
```

ส่วนของ PowerSum Task ใน index.h js เพื่อรับข้อมูลจากESP32

ส่วนของ Power Task ใน index.h js เพื่อรับข้อมูลจากESP32

```
222 function CTPower() {  
223     fetch("/CTPowerFlee")  
224     .then(response => response.text())  
225     .then(data => {  
226         document.getElementById("CTPowerFlee").textContent = data;  
227     });  
228 }
```

```
233 function PowFleePer() {  
234     fetch("/PowerFleePerHr")  
235     .then(response => response.text())  
236     .then(data => {  
237         document.getElementById("PowerFleePerHr").textContent = data;  
238     });  
239  
240     fetch("/PowerFleePerDay")  
241     .then(response => response.text())  
242     .then(data => {  
243         document.getElementById("PowerFleePerDay").textContent = data;  
244     });  
245 }  
246  
247 PowFleePer();  
248 setInterval(PowFleePer, 22600); // Update temperature every 4 seconds  
249 </script>
```


Test & Result

Home power usage monitor



Your room is now **Dark!** turn on the light?

Light switch is turn **OFF** ☐

Current Power Flee: **0.0091**THB



Temperature: **27.10°** Heat Index: **27.41°**

Your room is **COOL!**

Air conditioner is **OFF** ☐

Air Flee: **0.1550**THB



Power usage Flee: **9.1158**THB

Power Flee/Hour: **0.0000**THB

Power Flee/Day: **0.0000**THB

```
test2 / src / ... main.cpp ... lightTask(void)
Serial.println(lightStatus);
106 <1 class="fa fa-thermometer-ha

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

ESP32 Web Server: New request received:
GET /temperature
ESP32 Web Server: New request received:
GET /heatIndex
ESP32 Web Server: New request received:
GET /heatStatus
ESP32 Web Server: New request received:
GET /airFlee
ESP32 Web Server: New request received:
GET /airStatus
ESP32 Web Server: New request received:
GET /CTPowerFlee
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /PowerFleePerHr
1906.16 8.29
1809.31 7.87
PU:2.3020
PF:9.1158
6372
4095
Dark! turn on the light?

light is OFF
Humidity: 48.30% Temperature: 27.10 C Heat index: 27.41 C COOL!

air is OFF
ESP32 Web Server: New request received:
GET /light
ESP32 Web Server: New request received:
GET /lightFlee
ESP32 Web Server: New request received:
GET /LightSwitchStatus
ESP32 Web Server: New request received:
GET /temperature
ESP32 Web Server: New request received:
GET /heatIndex
ESP32 Web Server: New request received:
GET /heatStatus
ESP32 Web Server: New request received:
GET /airFlee
ESP32 Web Server: New request received:
GET /airStatus
ESP32 Web Server: New request received:
GET /CTPowerFlee
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /PowerFleePerHr
```

Test & Result

Home power usage monitor



Your room is now **Dark!** turn on the light?

Light switch is turn **ON** 

Current Power Flee: **0.0091**THB



Temperature: **27.00°** Heat Index: **27.30°**

Your room is **COOL!**

Air conditioner is **ON** 

Air Flee: **0.1550**THB



Power usage Flee: **10.0184**THB

Power Flee/Hour: **0.0000**THB

Power Flee/Day: **0.0000**THB

```
Serial.println(lightStatus);
106
<1 class="fa fa

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

ESP32 Web Server: New request received:
GET /airFlee
ESP32 Web Server: New request received:
GET /airStatus
ESP32 Web Server: New request received:
GET /CTPowerFlee
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /PowerFleePerHr
1752.42 7.62
1399.96 6.09
PU:2.5299
PF:10.0184
6372
4095
Dark! turn on the light?

light is OFF
Humidity: 48.00% Temperature: 27.00 C Heat index: 27.30 C COOL!

air is OFF
ESP32 Web Server: New request received:
GET /light
ESP32 Web Server: New request received:
GET /lightFlee
ESP32 Web Server: New request received:
GET /LightSwitchStatus
ESP32 Web Server: New request received:
GET /temperature
ESP32 Web Server: New request received:
GET /heatIndex
ESP32 Web Server: New request received:
GET /heatStatus
ESP32 Web Server: New request received:
GET /airFlee
ESP32 Web Server: New request received:
GET /airStatus
ESP32 Web Server: New request received:
GET /CTPowerFlee
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ON
ESP32 Web Server: New request received:
GET /LightSwitchStatus
ON
ESP32 Web Server: New request received:
GET /airStatus
```

Test & Result

Home power usage monitor



Your room is now **Bright!**

Light switch is turn **ON** 

Current Power Flee: **0.0137**THB



Temperature: **26.90°** Heat Index: **27.21°**

Your room is **COOL!**

Air conditioner is **OFF** 

Air Flee: **0.2325**THB



Power usage Flee: **10.7557**THB

Power Flee/Hour: **0.0000**THB

Power Flee/Day: **0.0000**THB

```
Serial.println(lightStatus);
106 <1 class="fa

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

GET /heatStatus
ESP32 Web Server: New request received:
GET /airFlee
ESP32 Web Server: New request received:
GET /airStatus
ESP32 Web Server: New request received:
GET /CTPowerFlee
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /PowerFleePerHr
3075.43 13.37
3017.60 13.12
PU:2.7161
PF:10.7557
6372
2705
Bright!

light is ON0.0034
0.0137
Humidity: 48.10% Temperature: 26.90 C Heat index: 27.21 C COOL!

air is ONAU:0.0587
AF:0.2325
ESP32 Web Server: New request received:
GET /light
ESP32 Web Server: New request received:
GET /lightFlee
ESP32 Web Server: New request received:
GET /LightSwitchStatus
ESP32 Web Server: New request received:
GET /temperature
ESP32 Web Server: New request received:
GET /heatIndex
ESP32 Web Server: New request received:
GET /heatStatus
ESP32 Web Server: New request received:
GET /airFlee
ESP32 Web Server: New request received:
GET /airStatus
ESP32 Web Server: New request received:
GET /CTPowerFlee
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /PowerFleePerHr
OFF
ESP32 Web Server: New request received:
GET /airStatus
```


Test & Result

Home power usage monitor



Your room is now **Bright!**

Light switch is turn **ON** 

Current Power Flee: **0.1047**THB



Temperature: **30.70°** Heat Index: **32.15°**

Your room is **so HOT!** turn on the cooler?

Air conditioner is **OFF** 

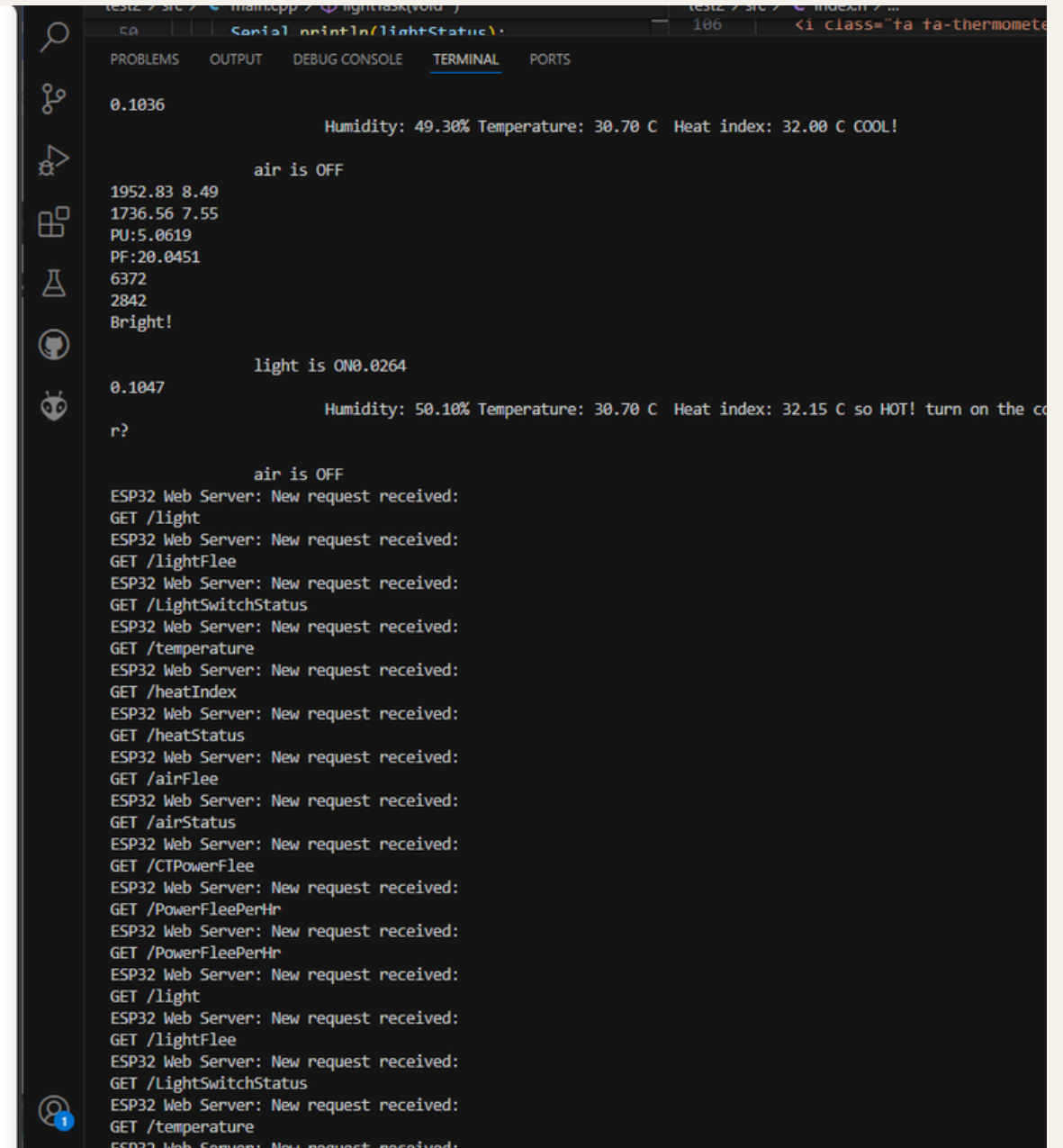
Air Flee: **0.2325**THB



Power usage Flee: **20.0451**THB

Power Flee/Hour: **19.0493**THB

Power Flee/Day: **0.0000**THB



```
0.1036
Humidity: 49.30% Temperature: 30.70 C Heat index: 32.00 C COOL!

air is OFF

1952.83 8.49
1736.56 7.55
PU:5.0619
PF:20.0451
6372
2842
Bright!

light is ON0.0264

0.1047
Humidity: 50.10% Temperature: 30.70 C Heat index: 32.15 C so HOT! turn on the co
r?

air is OFF
ESP32 Web Server: New request received:
GET /light
ESP32 Web Server: New request received:
GET /lightFlee
ESP32 Web Server: New request received:
GET /LightSwitchStatus
ESP32 Web Server: New request received:
GET /temperature
ESP32 Web Server: New request received:
GET /heatIndex
ESP32 Web Server: New request received:
GET /heatStatus
ESP32 Web Server: New request received:
GET /airFlee
ESP32 Web Server: New request received:
GET /airStatus
ESP32 Web Server: New request received:
GET /CTPowerFlee
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /light
ESP32 Web Server: New request received:
GET /lightFlee
ESP32 Web Server: New request received:
GET /LightSwitchStatus
ESP32 Web Server: New request received:
GET /temperature
ESP32 Web Server: New request received:
```

Test & Result

Home power usage monitor



Your room is now **Bright!**

Light switch is turn **ON** 

Current Power Flee: **0.1070THB**



Temperature: **30.80°** Heat Index: **32.40°**

Your room is

Air conditioner is **ON** 

Air Flee: **0.2325THB**



Power usage Flee: **20.2422THB**

Power Flee/Hour: **19.0493THB**

Power Flee/Day: **0.0000THB**

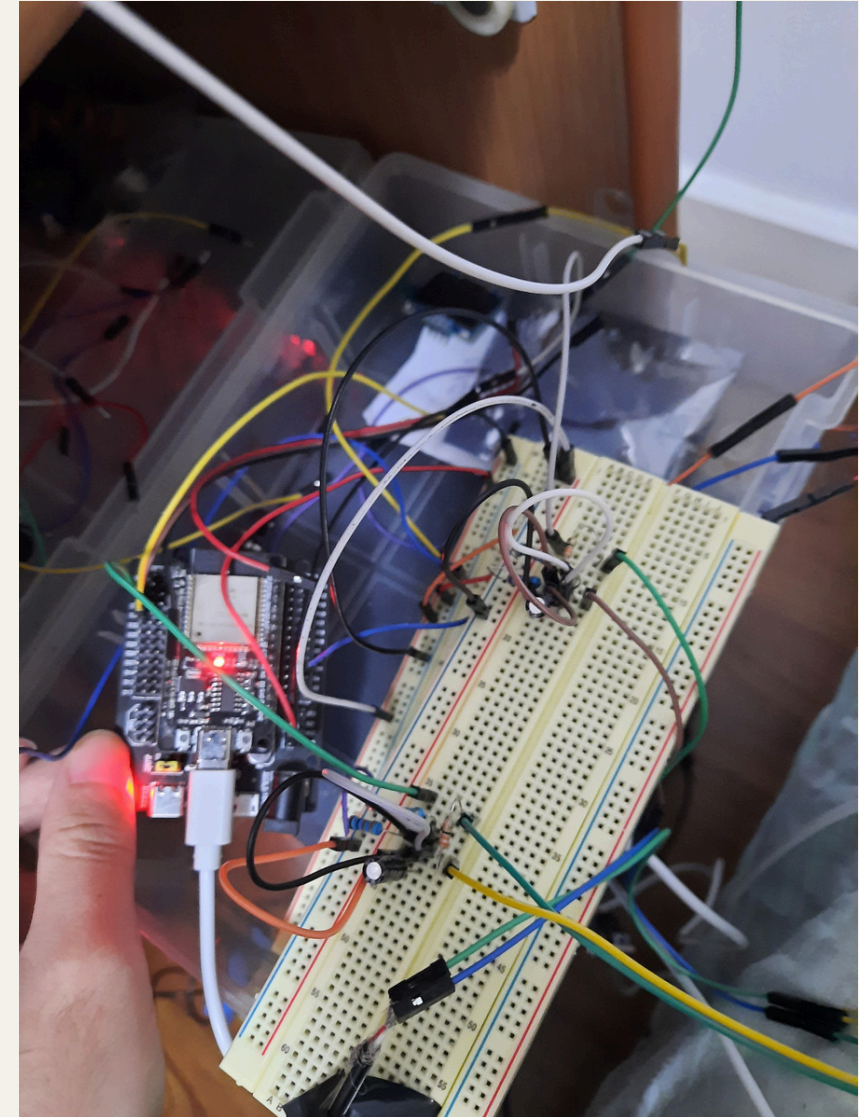
```
Serial.println(lightStatus);
106
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

GET /heatStatus
ESP32 Web Server: New request received:
GET /airFlee
ESP32 Web Server: New request received:
GET /airStatus
ESP32 Web Server: New request received:
GET /CTPowerFlee
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ON
ESP32 Web Server: New request received:
GET /airStatus
2989.17 13.00
1471.83 6.40
PU:5.1117
PF:20.2422
6372
2832
Bright!

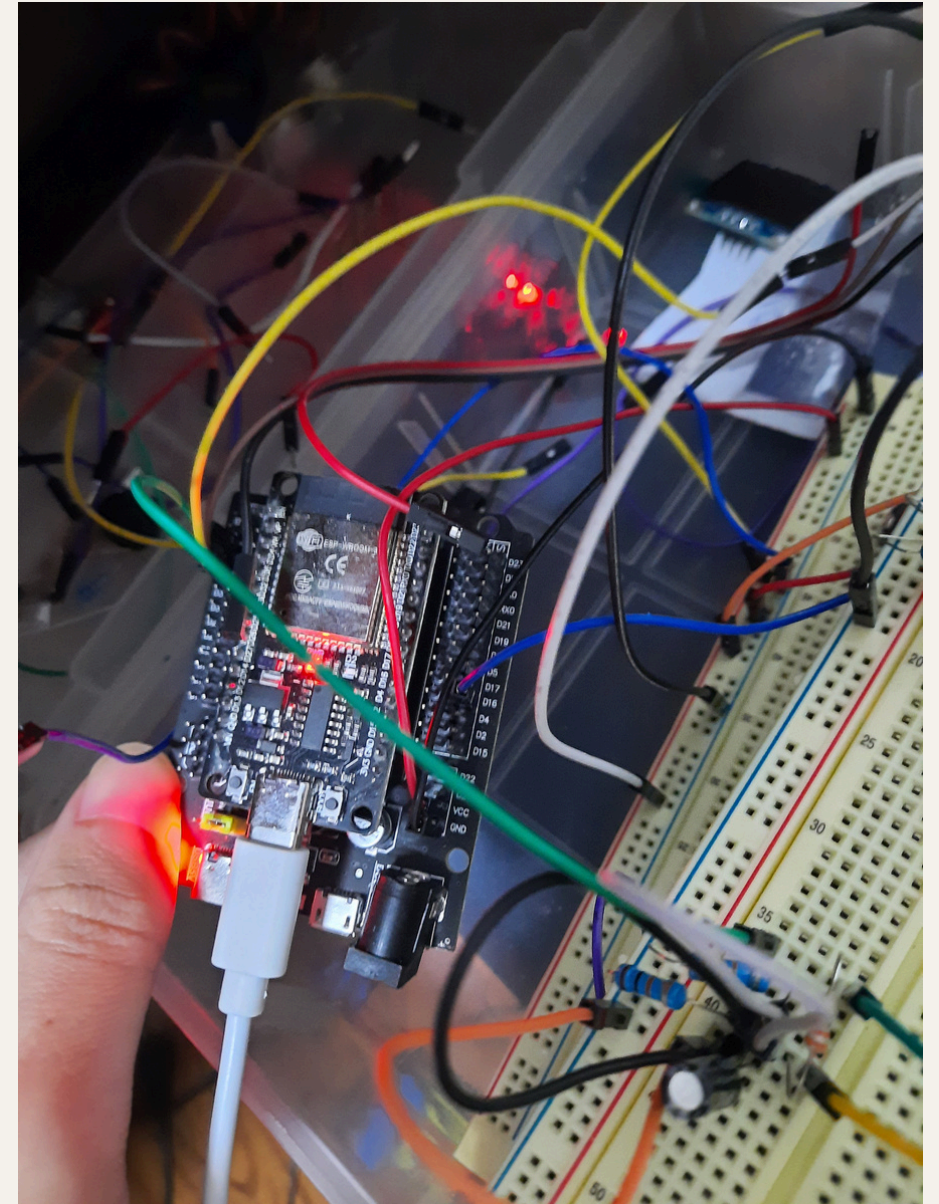
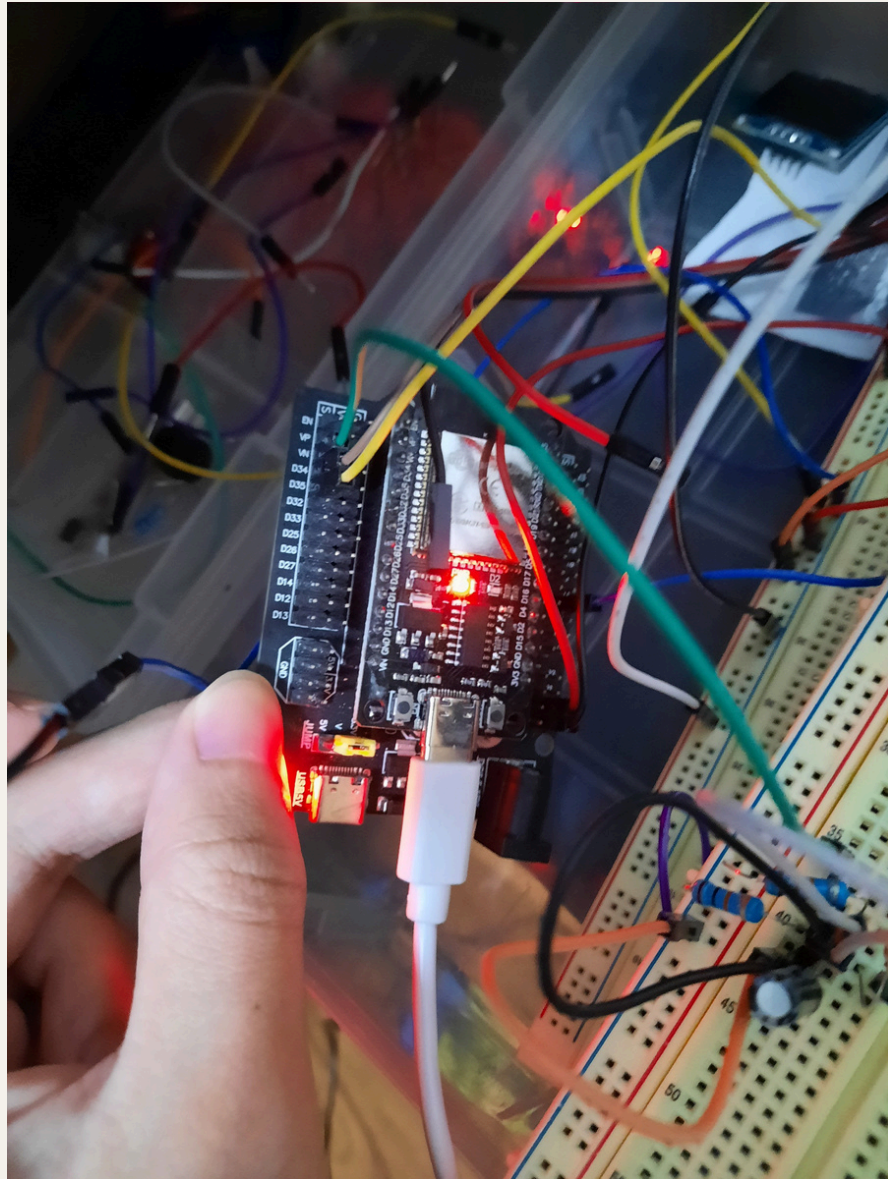
light is ON0.0270
0.1070
Humidity: 50.60% Temperature: 30.80 C Heat index: 32.40 C

air is ONAU:0.0587
AF:0.2325
ESP32 Web Server: New request received:
GET /light
ESP32 Web Server: New request received:
GET /lightFlee
ESP32 Web Server: New request received:
GET /LightSwitchStatus
ESP32 Web Server: New request received:
GET /temperature
ESP32 Web Server: New request received:
GET /heatIndex
ESP32 Web Server: New request received:
GET /heatStatus
ESP32 Web Server: New request received:
GET /airFlee
ESP32 Web Server: New request received:
GET /airStatus
ESP32 Web Server: New request received:
GET /CTPowerFlee
ESP32 Web Server: New request received:
GET /PowerFleePerHr
ESP32 Web Server: New request received:
GET /PowerFleePerHr
```


Project Picture



Project Picture



VDO Demonstration

Home power usage monitor

💡

Your room is now **Dark!** turn on the light?

Light switch is turn **OFF** ☐

Current Power Flee: **0.0000**THB

🌡️

Temperature: **30.30°** Heat Index: **32.03°**

Your room is **so HOT!** turn on the cooler?

Air conditioner is **OFF** ☐

Air Flee: **0.0000**THB

⚡

Power usage Flee: **3.7942**THB

Power Flee/Hour: **0.0000**THB

Power Flee/Day: **0.0000**THB

```
Writing at 0x0008e121... (65 %)
Writing at 0x0009376f... (68 %)
Writing at 0x00098eff... (71 %)
Writing at 0x0009ee49... (75 %)
Writing at 0x000a46d9... (78 %)
Writing at 0x000ab872... (81 %)
Writing at 0x000b5566... (84 %)
Writing at 0x000ba761... (87 %)
Writing at 0x000c36f8... (90 %)
Writing at 0x000c93d6... (93 %)
Writing at 0x000ceb9a... (96 %)
Writing at 0x000d42f7... (100 %)
Wrote 815328 bytes (515924 compressed) at 0x00010000 in 12.2 seconds (effective 536.7 kbit/s).
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
===== [SUCCESS] Took 22.46 seconds =====
Terminal will be reused by tasks, press any key to close it.

Executing task in folder test2: C:\Users\User\.platformio\penv\Scripts\platformio.exe run --t
get upload

Processing esp32doit-devkit-v1 (platform: espressif32; board: esp32doit-devkit-v1; framework: ardu
no)
-----
Verbose mode can be enabled via '-v, --verbose' option
CONFIGURATION: https://docs.platformio.org/page/boards/espressif32/esp32doit-devkit-v1.html
PLATFORM: Espressif 32 (6.9.0) > DOIT ESP32 DEVKIT V1
HARDWARE: ESP32 240MHz, 320KB RAM, 4MB Flash
DEBUG: Current (cmsis-dap) External (cmsis-dap, esp-bridge, esp-prog, iot-bus-jtag, jlink, minimodu
le, olimex-arm-usb-ocd, olimex-arm-usb-ocd-h, olimex-arm-usb-tiny-h, olimex-jtag-tiny, tumpa)
PACKAGES:
- framework-arduinoespressif32 @ 3.20017.0 (2.0.17)
- tool-esptoolpy @ 1.40501.0 (4.5.1)
- tool-mkfatfs @ 2.0.1
- tool-mklittlefs @ 1.203.210628 (2.3)
- tool-mkspiffs @ 2.230.0 (2.30)
- toolchain-xtensa-esp32 @ 8.4.0+2021r2-patch5
LDF: Library Dependency Finder -> https://bit.ly/configure-pio-ldf
LDF Modes: Finder ~ chain, Compatibility ~ soft
Found 38 compatible libraries
Scanning dependencies...
Dependency Graph
|-- DHT sensor library @ 1.4.6
|-- EmonLib @ 1.1.0
|-- ESPAsyncWebServer-esphome @ 3.3.0
|-- Adafruit Unified Sensor @ 1.1.15
|-- WiFi @ 2.0.0
Building in rele
PlatformIO: Upload
```

Problem & Solution

```
light is OFF
Guru Meditation Error: Core  0 panic'ed (Interrupt wdt timeout on CPU0).

Core  0 register dump:
PC      : 0x400d4655  PS      : 0x00060235  A0      : 0x800d4740  A1      : 0x3ffd38c0
A2      : 0x0001e662  A3      : 0x00000001  A4      : 0x3ffc3dd0  A5      : 0x00060023
A6      : 0x00060023  A7      : 0x00000001  A8      : 0x00027100  A9      : 0x0001e662
A10     : 0x00000010  A11     : 0x00000000  A12     : 0x782fe6c6  A13     : 0x00010000
A14     : 0x00000028  A15     : 0x003fffff  SAR     : 0x00000010  EXCCAUSE: 0x00000005
EXCVADDR: 0x00000000  LBEG    : 0x40084625  LEND    : 0x4008462d  LCOUNT   : 0x00000026

Backtrace: 0x400d4652:0x3ffd38c0 0x400d473d:0x3ffd38e0 0x400d4915:0x3ffd3a40 0x400d3a0e:0x3ffd3a60

ELF file SHA256: 3cedb9da4fd29d59

Rebooting...
???D?8 11Connecting to WiFi...
Connecting to WiFi...
Connected to WiFi
```



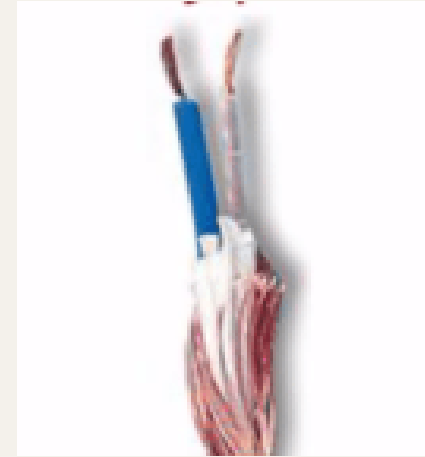
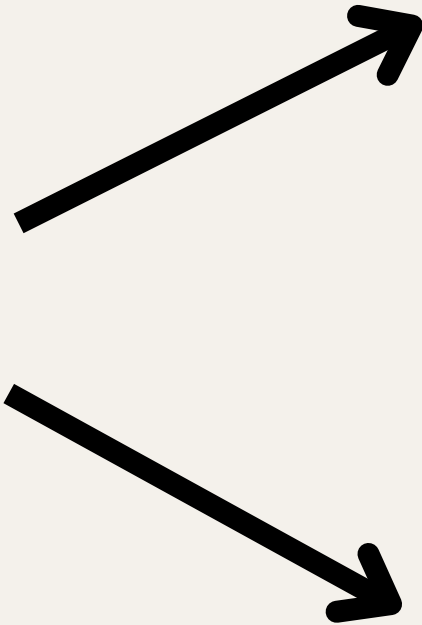
use Mutex

**sensors can not read value when ESP32
connected to WIFI**

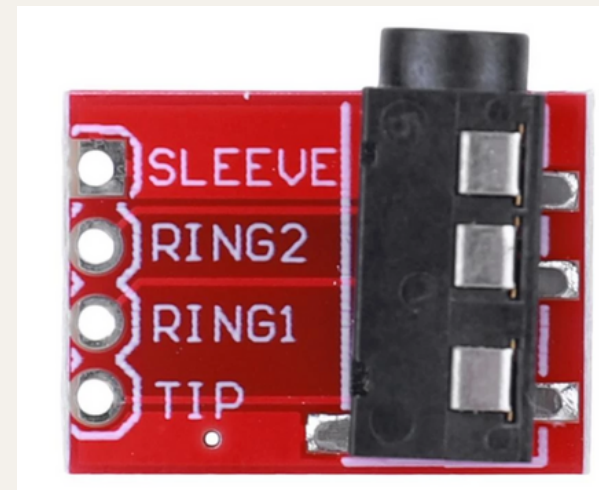


**use ADC, INPUT_ONLY
pin(GPIO34, 35, 36, 39)**

Problem & Solution



cut jack out

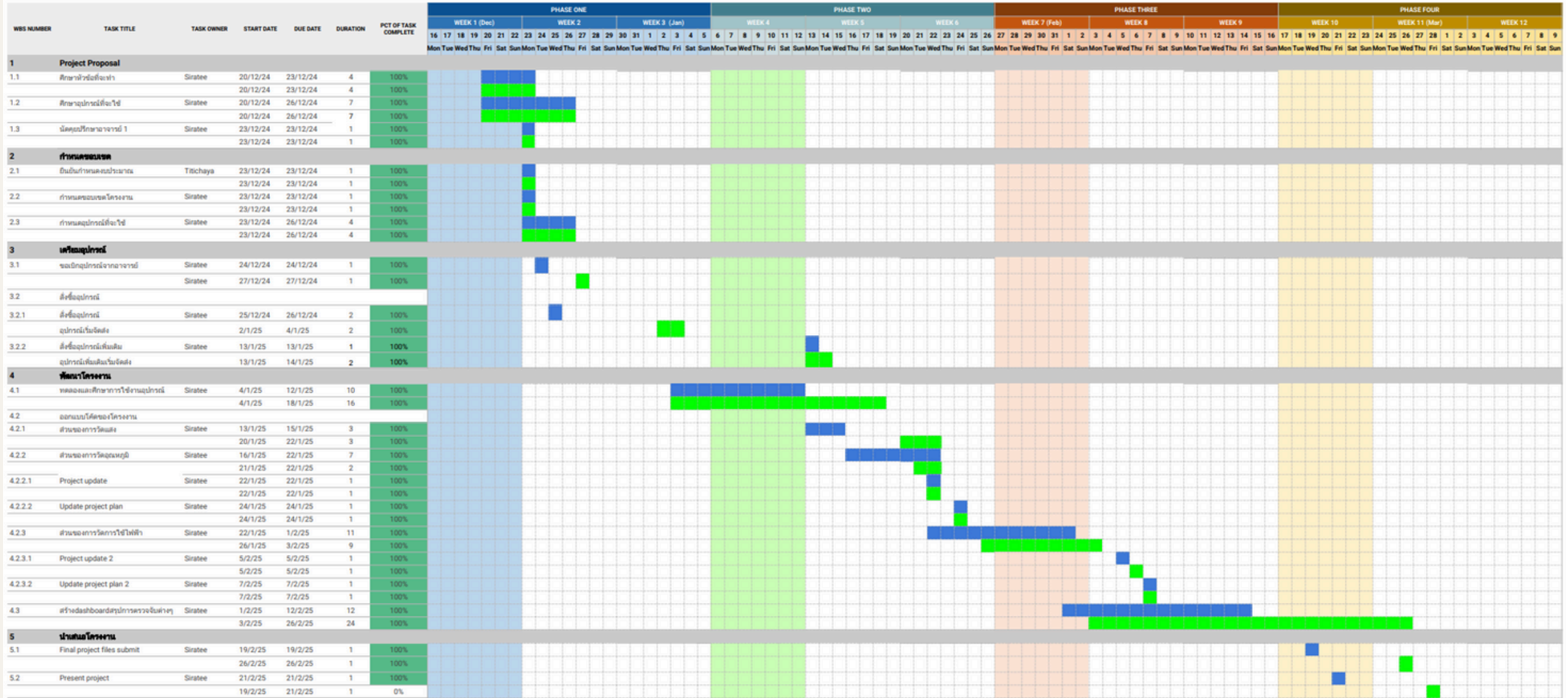


**use jack
breakout**

GANTT CHART TEMPLATE

A Gantt chart's visual timeline allows you to see details about each task as well as project dependencies.

 กำหนดการ
 เสร็จจริง



Special Thanks

Producer & Advisor

Titichaya Thanamitsomboon

Supplies Supporter

Jirayu Peetakul

Grisnai Chittinan

CyberTice

Arduitrronics

Electronics Source

C 203

Read more...

